

LogicConnect: Structured Connectivity Design for Differentiable Logic Gate Networks

Abstract—Differentiable logic gate networks learn Boolean circuits whose connectivity determines which signals can be combined during gate-function optimization. Learning these connections can improve accuracy, but increases the memory required to process candidate inputs during training. In this paper, we present LogicConnect, a structured connectivity method that constructs the graph before training while preserving the network’s gate budget. LogicConnect combines semantic input pairing with regular multiscale matchings in deeper layers and supports dense and convolutional architectures without trainable routing parameters. We evaluate the method on MNIST, Fashion-MNIST, and CIFAR-10 through accuracy comparisons, training-resource measurements, and structural ablations. LogicConnect improves dense CIFAR-10 test accuracy by 3.18–4.59 percentage points across three model sizes and three paired seeds. Convolutional S/M models improve by 3.26/2.08 points in single-seed tests, although the three-seed S validation comparison remains inconclusive. Compared with learned Top-32 routing, LogicConnect uses 93.7–94.2% less peak GPU memory and 80% fewer trainable parameters, with an accuracy reduction of 1.84–5.30 points. Degree-preserving ablations identify first-layer pairing as the strongest measured dense contribution, supporting structured connectivity as a design choice for improving accuracy under constrained training memory.

I. INTRODUCTION

Differentiable Logic Gate Networks (DLGNs) provide an alternative path toward efficient machine learning at the edge by learning directly with the primitive operations used in digital hardware. Rather than relying on multiply-accumulate operations, a DLGN is composed of Boolean gates. During training, these discrete operations are represented through differentiable relaxations, allowing the network to be optimized with gradient descent. After training, each relaxed gate is hardened to a Boolean function for efficient inference [1]. This direct correspondence between the learned model and Boolean computation makes DLGNs particularly attractive for resource-constrained embedded and edge and systems.

However, a fundamental characteristic of DLGNs is their sparse connectivity. Each conventional DLGN gate has only two inputs. These two predecessors are selected before training and remain fixed, while optimization determines which Boolean function the gate should implement [1]. Therefore, training can change *how* two signals are combined, but not *which* signals arrive at a gate. To that end, if two informative signals are never connected through the available graph, learning the gate

functions alone cannot create that missing edge. At a fixed depth and width, connectivity consequently determines the combinations of information that can be formed throughout the network and becomes part of the effective architectural capacity of the DLGN.

A way to address this issue is to consider connectivity as an independent design dimension [2] and optimize the predecessors themselves during training [3], [4]. This type of learned connectivity can recover connections that were absent from the original graph and can therefore improve accuracy. However, such methods require significantly increased training time as each gate must consider multiple candidate signals, requiring additional routing state and candidate activations even though the final hardened network may retain only a small number of predecessors. This creates a practical design choice between inexpensive but arbitrary *fixed random connectivity* and more flexible but substantially heavier *learned connectivity*.

To demonstrate the need for connectivity design, we consider a motivational example on a dense CIFAR-10 DLGN. We keep the deeper connectivity unchanged and randomize only the first-layer connections under a degree-preserving constraint. Hence, every predecessor is used exactly the same number of times before and after randomization; only the identities of the signals paired at each gate are changed. Despite preserving the same fan-out, hardened validation accuracy decreases by 4.49 percentage points. This result reveals an important limitation of degree-balanced connectivity: *how often a signal is used does not determine which information is brought together*. However, explicitly selecting favorable pairs is itself challenging to scale. Given N candidate predecessors, a two-input gate can already choose from $O(N^2)$ possible pairs, and this choice is repeated across thousands or millions of gates and multiple network layers. Moreover, while input signals have explicit image semantics, intermediate signals progressively lose this direct interpretation, making a manually defined pairing rule increasingly difficult to justify at deeper layers. Fixed random connectivity avoids this search but ignores signal relationships, whereas learned connectivity explores the enlarged space by maintaining additional routing state and candidate activations during training. Therefore, a key challenge is to construct informative connections *before training* using principles that remain scalable as the network grows and do not introduce the training overhead of a separate routing problem.

In this paper, we propose **LogicConnect**, an offline connectivity construction that introduces structure into the DLGN graph before training. LogicConnect first exploits signal identity where it is most explicit by forming semantic first-layer pairs

according to relationships among the encoded inputs rather than arbitrary indices. In subsequent layers, it organizes predecessor interactions through regular multiscale matchings, selecting complete matching stages primarily to balance predecessor usage and using ancestry overlap to distinguish alternatives with equivalent degree balance. Our main contributions are:

II. RELATED WORK

Petersen et al. introduced Differentiable Logic Gate Networks (DLGNs), enabling gradient-based optimization of Boolean gate functions while keeping randomly initialized connections fixed during training [1]. In [5], the authors extended DLGNs to convolutional architectures, exploiting image structure through shared logic trees, spatial sampling, pooling, and residual initialization. Other works extend DLGNs to recurrent models [6], improve output aggregation for larger classification problems [7], or develop interpretable and truth-table-based logic architectures [8]–[10]. Related differentiable logic models have also been explored for cellular automata [11], [12] and circuit representation learning [13]. However, these works primarily focus on extending the architecture and applicability of logic-based networks, without exploring the connectivity problem.

Regarding DLGN connectivity, Mommen et al. optimize distributions over candidate predecessors, allowing gate inputs to change during training [3]. Similarly, LILogic Net employs sparse Top-K routing to learn compact network connectivity [4], while BitLogic studies connectivity together with encoding, fan-in, node parameterization, and output-head design [2]. These works demonstrate that selecting better predecessors can significantly improve logic networks. However, learned routing requires processing candidate connections and maintaining additional routing parameters and intermediate activations during training, increasing both execution time and memory requirements, even though the hardened network ultimately retains only a small number of inputs per gate. In contrast, our approach constructs the connectivity *offline* and keeps it fixed during gate training, avoiding an additional routing optimization problem while preserving the declared depth, width, and gate budget.

Additionally, several works focus on improving the optimization of the gate functions themselves. Mind the Gap reduces the discrepancy between continuous training and discrete inference [14]. Light DLGNs redesign the gate parameterization to improve training efficiency [15], while WARP introduces a Walsh-based Boolean representation [16]. The works in [17], [18] further explore polynomial formulations to analyze or replace conventional soft gate mixtures. These methods optimize *what function* a gate computes from its assigned inputs, whereas our work focuses on *which signals* are made available to that gate.

REFERENCES

- [1] F. Petersen, C. Borgelt, H. Kuehne, and O. Deussen, "Deep differentiable logic gate networks," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 2006–2018.
- [2] S. Bühner, A. Plesner, T. Aczel, and R. Wattenhofer, "BitLogic: A framework for gradient-based LUT-native neural networks," *Transactions on Machine Learning Research*, Jun. 2026.
- [3] W. Mommen, L. Keuninckx, M. Hartmann, and P. Wambacq, "A method for optimizing connections in differentiable logic gate networks," 2025.
- [4] K. Fojcik, R. Zioma, and J. Armitis, "LILogic Net: Compact logic gate networks with learnable connectivity for efficient hardware deployment," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2026, pp. 3498–3507.
- [5] F. Petersen, H. Kuehne, C. Borgelt, J. Welzel, and S. Ermon, "Convolutional differentiable logic gate networks," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [6] S. Bühner, A. Plesner, T. Aczel, and R. Wattenhofer, "Recurrent deep differentiable logic gate networks," in *Proceedings of the 2nd International Workshop on Edge and Mobile Foundation Models*, 2025, pp. 31–36.
- [7] S. Brändle, T. Aczel, A. Plesner, and R. Wattenhofer, "From MNIST to ImageNet: Understanding the scalability boundaries of differentiable logic gate networks," 2025.
- [8] C. Yue and N. K. Jha, "Learning interpretable differentiable logic networks," *IEEE Transactions on Circuits and Systems for Artificial Intelligence*, vol. 1, no. 1, pp. 69–82, 2024.
- [9] A. Benamira, T. Guérard, T. Peyrin, T. Yap, and B. Hooi, "A scalable, interpretable, verifiable & differentiable logic gate convolutional neural network architecture from truth tables," 2022.
- [10] A. Benamira, T. Peyrin, T. Yap, T. Guérard, and B. Hooi, "Truth table net: Scalable, compact & verifiable neural networks with a dual convolutional small boolean circuit networks form," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, 2024, pp. 13–21.
- [11] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin, "Growing neural cellular automata," *Distill*, vol. 5, no. 2, 2020.
- [12] P. Miotti, E. Niklasson, E. Randazzo, and A. Mordvintsev, "Differentiable logic cellular automata: From game of life to pattern generation," in *ALIFE 2025: Proceedings of the Artificial Life Conference 2025*, 2025.
- [13] M. Li, S. Khan, Z. Shi, N. Wang, H. Yu, and Q. Xu, "Deepgate: Learning neural representations of logic gates," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 667–672.
- [14] S. Yousefi, A. Plesner, T. Aczel, and R. Wattenhofer, "Mind the gap: Removing the discretization gap in differentiable logic gate networks," in *Advances in Neural Information Processing Systems*, vol. 38, 2025, pp. 135 798–135 828.
- [15] L. Rüttgers, T. Aczel, A. Plesner, and R. Wattenhofer, "Light differentiable logic gate networks," in *International Conference on Learning Representations*, 2026.
- [16] L. Gerlach, T. Gerlach, L. Våge, E. Kauffman, and I. Ojalvo, "WARP logic neural networks," 2026.
- [17] Y. Kim, "Fitting multilinear polynomials for logic gate networks," 2026.
- [18] S. S. Damera, R. Matheu, A. G. Puranic, and J. S. Baras, "Polynomial surrogate training for differentiable ternary logic gate networks," 2026.